

Vendredi 6/03/2026

Rapport Back-end Partie : Trajets

Le dossier **Trajets** est responsable de la **gestion complète des trajets de covoiturage** sur la plateforme ShareTrajet.

Il permet notamment :

- la **création et publication de trajets** par les conducteurs,
- la **recherche de trajets** par les passagers,
- la **gestion des étapes d'un trajet**,
- le **calcul des prix et commissions de la plateforme**,
- la **gestion des préférences et options du trajet**,
- l'**administration des trajets et des prix du carburant**.

Dossier migrations

Il est utilisé par Django pour gérer les **modifications de la base de données**.

Quand on crée ou modifie un **modèle (models.py)**, Django génère automatiquement un fichier de migration pour **créer ou mettre à jour les tables dans la base de données**, il contient des fichiers suivants:

__init__.py :

Ce fichier est **vide**.

Son rôle est simplement de dire à Python que **le dossier migrations est un module Python** afin que Django puisse l'utiliser.

0001_initial.py :

C'est la **première migration du projet**.

Elle sert à **créer les tables principales dans la base de données**, comme : **Trajet**, **FuelPrice**

Ce fichier contient aussi :

- les **relations entre les tables** (ForeignKey, ManyToMany)
- les **contraintes et validations**
- les **index pour optimiser les recherches**

1. __init__.py

un fichier standard de Python permettant d'indiquer que le dossier **trajets** est un **module Python**.

Même s'il est vide, il permet à Django de :

- reconnaître le dossier comme une **application Python**,
- permettre l'importation des fichiers du module (models, views, serializers etc.).

Sans ce fichier, certains imports internes du projet pourraient ne pas fonctionner correctement.

2. admin.py

Le fichier **admin.py** configure la gestion des modèles dans **l'interface d'administration Django**.

Cette interface permet aux administrateurs de :

- consulter les trajets,
- modifier leur statut,
- gérer les étapes,
- gérer les prix du carburant.

Modèles administrés

2.1 TrajetAdmin

Permet d'administrer les trajets publiés par les conducteurs.

Fonctionnalités principales :

- Affichage des informations importantes :
 - ville de départ et d'arrivée
 - conducteur
 - date et heure
 - prix
 - places disponibles
 - statut du trajet
- Filtres :
 - statut
 - date
 - options confort
 - pause obligatoire
- Recherche :
 - email du conducteur
 - nom du conducteur
 - villes du trajet

Fonctions supplémentaires :

- `route()` : Affiche le trajet sous forme
Ville départ → Ville arrivée

- `places_info()` : Affiche les places sous forme **places disponibles / places totales**
- `status_badge()` : Affiche le statut avec une **couleur visuelle** :
 - vert : actif
 - bleu : terminé
 - rouge : annulé

Actions administratives :

- Annuler plusieurs trajets
- Marquer plusieurs trajets comme terminés

2.2 TrajetEtapeAdmin

Permet de gérer les étapes intermédiaires d'un trajet.

Par exemple :

Alger → **Bouira** → **Béjaïa**

Informations visibles :

- trajet
- ordre de l'étape
- Ville
- heure d'arrivée

2.3 FuelPriceAdmin

Permet de gérer les **prix du carburant**.

Ces données servent à :

- calculer le **prix conseillé d'un trajet**
- adapter le prix selon la **wilaya et le type de carburant**

3. models.py

Le fichier **models.py** définit la structure des données stockées dans la base de données.

Plusieurs modèles représentent les entités principales du système de covoiturage.

3.1 Modèle Trajet

Le modèle **Trajet** représente un **trajet de covoiturage publié par un conducteur**.

Informations stockées :

Conducteur

Relation avec l'utilisateur :

conducteur → User

Cela permet d'identifier **qui propose le trajet**.

Informations du trajet

- ville de départ
 - ville d'arrivée
 - adresse de départ
 - adresse d'arrivée
 - date
 - heure de départ
-

Capacité du véhicule

- nombre total de places
- nombre de places encore disponibles

Ces informations permettent au système de **gérer les réservations**.

Tarification

Chaque trajet contient :

- **price** : prix fixé par le conducteur
 - **price_platform** : commission de la plateforme
 - **price_driver** : montant reversé au conducteur
 - **suggested_price** : prix conseillé calculé automatiquement
-

Distance et options

- distance du trajet
 - option **trajet confort**
 - pause obligatoire si trajet long
-

Carburant

Le modèle prend en compte :

- type de carburant
- consommation du véhicule

Ces informations servent à **calculer un prix conseillé**.

Préférences

Le conducteur peut définir certaines préférences :

- musique
- non-fumeur
- animaux autorisés

Ces préférences sont liées au trajet via une **relation ManyToMany**.

Statut du trajet

Un trajet peut être :

- **ACTIVE** : disponible
 - **COMPLETED** : terminé
 - **CANCELLED** : annulé
-

Méthodes importantes

save() : Cette méthode est redéfinie pour effectuer automatiquement certains calculs :

- calcul de la **commission de la plateforme**
- calcul du **revenu du conducteur**
- ajout du **supplément confort**
- activation de la **pause obligatoire** si distance > seuil

get_total_price_for_passenger() : Calcule le **prix total payé par un passager**.

can_reserve() : Vérifie si un passager peut réserver :

- trajet actif
- suffisamment de places disponibles

update_disponibilite() : Met à jour le nombre de **places disponibles** en fonction des réservations confirmées.

3.2 Modèle TrajetPreference

C'est une **table intermédiaire** entre :

Trajet ↔ Preference

Elle permet d'associer plusieurs préférences à un trajet.

Exemple :

Trajet :

Alger → Oran

Préférences :

- non-fumeur
- musique autorisée

3.3 Modèle TrajetEtape

Ce modèle représente les **étapes intermédiaires d'un trajet**.

Exemple :

Alger → Bouira → Béjaïa

Chaque étape contient :

- ville
- adresse
- heure d'arrivée
- ordre de l'étape

3.4 Modèle FuelPrice

Ce modèle stocke les **prix du carburant par wilaya**.

Informations stockées :

- code de wilaya
- nom de la wilaya
- type de carburant
- prix par litre
- date de mise à jour

Utilité dans le projet

Ces données servent à :

- estimer le **coût réel d'un trajet**
- calculer le **prix conseillé du covoiturage**

4. *permissions.py*

Ce fichier contient les **permissions personnalisées de l'API**.

Les permissions contrôlent **qui peut faire quoi sur les trajets**.

4.1 **IsDriverOrReadOnly**

Cette permission autorise :

- tout utilisateur authentifié à **consulter les trajets**
- seulement le **conducteur du trajet** à le modifier

4.2 **CanModifyTrajet**

Cette permission empêche :

- un utilisateur autre que le conducteur de modifier le trajet

- de modifier un trajet **terminé ou annulé**
 - de modifier certaines informations si **des réservations sont confirmées**
-

4.3 CanCancelTrajet

Permet uniquement au **conducteur** :

- d'annuler son trajet

Mais pas si :

- le trajet est déjà annulé
 - le trajet est déjà terminé
-

4.4 CanViewTrajetReservations

Contrôle l'accès aux **réservations du trajet**.

- Le conducteur peut voir **toutes les réservations**
- Les passagers peuvent voir **seulement leurs propres réservations**

5. serializers.py

Les **serializers** convertissent les données :

- de la **base de données** → **JSON**
- de **JSON** → **objets Django**

Ils sont essentiels pour les **API REST**.

5.1 TrajetCreateSerializer

Utilisé lors de la **création d'un trajet**.

Fonctionnalités :

- validation des données
 - vérification que le conducteur a une **CNI vérifiée**
 - calcul automatique du **prix conseillé**
 - création des **étapes**
 - association des **préférences**
-

5.2 TrajetListSerializer

Utilisé pour afficher **la liste des trajets**.

Il retourne les informations principales :

- conducteur
 - villes
 - date
 - prix
 - places disponibles
 - options
-

5.3 TrajetDetailSerializer

Utilisé pour afficher **le détail complet d'un trajet**.

Il inclut :

- informations du conducteur
 - préférences
 - étapes
 - passagers ayant réservé
 - nombre total de réservations
-

5.4 TrajetUpdateSerializer

Utilisé pour **modifier un trajet**.

Il vérifie notamment que :

- le nombre de places ne devient pas inférieur aux réservations déjà faites.
-

5.5 TrajetSearchSerializer

Permet d'effectuer une **recherche intelligente de trajets**.

Filtres disponibles :

- ville de départ
- ville d'arrivée
- date
- nombre de places
- prix maximum
- type de carburant
- préférences

- période de la journée
-

5.6 FuelPriceSerializer

Permet de manipuler les données de **prix du carburant** via l'API.

6. urls.py

Ce fichier définit les **routes de l'API REST** pour le module trajets.

Il utilise **Django REST Framework Router**.

7. Views.py :

Ce fichier contient les **API (endpoints) du backend Django REST** qui permettent de **gérer les trajets dans l'application ShareTrajet**.

Il définit deux ViewSets principaux :

- **TrajetViewSet** → gère toutes les opérations liées aux trajets
- **FuelPriceViewSet** → gère les prix du carburant

Dans Django REST, un **ViewSet** sert à :

- recevoir les requêtes HTTP du frontend
- accéder aux données dans la base
- renvoyer les réponses JSON

Conclusion :

Grâce à l'utilisation de **Django et Django REST Framework**, ce module fournit une **API robuste et sécurisée** permettant au frontend d'interagir efficacement avec les données des trajets.

Oudjane Anias 2CP7